ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 4

TECHNICAL PROGRAMMING

PAPER 2 · PRACTICE D · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central NC(V) Level 4 Technical Programming paper 2 · practice d.

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 4. It is not a Department of Higher Education and Training marking guideline.

How to mark: each ✓ is one mark. Programs are not marked strictly on syntax: give marks for correct logic, structure and output. Accept other names, messages and correct ways of solving the problem.

QUESTION 1

1.1 File named correctly with a comment on the first line ✓ (1)

1.2 class Part: ✓; def __init__(self, name, code, price, quantity=0): ✓✓; self.name and self.code ✓, self.price ✓, self.quantity ✓ (6)

1.3 def add_stock(self, n): ✓; if n > 0 ✓ quantity increased ✓; else "Invalid quantity" ✓ (4)

1.4 def remove_stock(self, n): ✓; if n > self.quantity ✓ "Not enough stock" ✓; else ✓ quantity reduced ✓ and new quantity displayed ✓ (6)

1.5 def value(self): ✓ return ✓ self.quantity * self.price ✓ (3)

1.6 class ElectronicPart(Part): ✓; constructor with voltage ✓ calling super().__init__(…) ✓✓; self.voltage ✓; def is_safe(self, robot_voltage): ✓ returns self.voltage <= robot_voltage (True / False) ✓ (7)

1.7 Object created with the given values ✓; 3 removed and the value displayed (R318.50) ✓; is_safe(12) displayed (True) ✓ (3)

```
# YourName - Question 1
class Part:
    def __init__(self, name, code, price, quantity=0):
        self.name = name
        self.code = code
        self.price = price
        self.quantity = quantity

    def add_stock(self, n):
        if n > 0:
            self.quantity += n
        else:
            print("Invalid quantity")

    def remove_stock(self, n):
        if n > self.quantity:
            print("Not enough stock")
        else:
            self.quantity -= n
            print(self.name, "now has", self.quantity, "in stock")

    def value(self):
        return self.quantity * self.price

class ElectronicPart(Part):
    def __init__(self, name, code, price, quantity, voltage):
        super().__init__(name, code, price, quantity)
        self.voltage = voltage

    def is_safe(self, robot_voltage):
        return self.voltage <= robot_voltage

sensor = ElectronicPart("Ultrasonic sensor", "US01", 45.50, 10, 5)
```

```
sensor.remove_stock(3)
print("Stock value: R", round(sensor.value(), 2))   # 7 x 45.50 = 318.50
print("Safe for a 12 V robot:", sensor.is_safe(12))
```

[30]

QUESTION 2

2.1 File named correctly ✓ (1)

2.2 def log_job(robot, job, minutes): ✓; open("jobs.csv", "a") ✓ (append mode, with or close) ✓; writes the three values separated by commas ✓ with a newline ✓ (5)

2.3 while loop ✓; robot name input and stop on "done" ✓; job input ✓; minutes changed with int() inside try ✓; except ValueError with the message ✓; log_job() called ✓ (6)

2.4 def totals(): ✓; empty dictionary ✓; try / open in read mode ✓; loop over the lines ✓; strip and split(",") ✓; minutes changed to int ✓; added to the robot's total (new key or +=) ✓✓; except FileNotFoundError with message and return {} ✓ (9)

2.5 totals() called ✓; loop that displays each robot and its minutes ✓✓; robot with the most minutes found with max() or a loop ✓✓ and displayed ✓ (6)

2.6 Program saved ✓, printed ✓ and a screenshot of the output ✓ (3)

```
# YourName - Question 2
def log_job(robot, job, minutes):
    with open("jobs.csv", "a") as f:
        f.write(f"{robot},{job},{minutes}\n")

while True:
    robot = input("Robot (or done): ")
    if robot.lower() == "done":
        break
    job = input("Job: ")
    try:
        minutes = int(input("Minutes: "))
    except ValueError:
        print("Minutes must be a number")
        continue
    log_job(robot, job, minutes)

def totals():
    result = {}
    try:
        with open("jobs.csv") as f:
            for line in f:
                robot, job, minutes = line.strip().split(",")
                result[robot] = result.get(robot, 0) + int(minutes)
    except FileNotFoundError:
        print("No jobs logged yet")
    return result

t = totals()
for robot, mins in t.items():
    print(robot, "-", mins, "minutes")
if t:
    busiest = max(t, key=t.get)
    print("Most minutes:", busiest)
```

[30]

QUESTION 3

3.1 File named correctly ✓ (1)

3.2 scores = { … } ✓ with the three teams ✓ and their lists of scores ✓ (3)

3.3 def add_team(): ✓; input ✓; if already in scores – message ✓; else scores[team] = [] ✓ (4)

3.4 def add_score(): ✓; team input ✓; not registered – message and return ✓; while loop ✓; int() inside try ✓; except ValueError message ✓; range check 0–100 ✓ with message ✓; append to the team's list and "Score saved." ✓ (9)

3.5 def leaderboard(): ✓; sorted() on the teams ✓ by the sum of the scores ✓ in reverse (highest first) ✓; numbered with enumerate or a counter ✓; total ✓ and number of runs with len() ✓ displayed (7)

3.6 def team_average(): ✓; not registered message ✓; no scores message ✓; sum ÷ len ✓ rounded to one decimal and displayed ✓ (5)

3.7 while True loop ✓; menu displayed and choice input ✓; choices 1 ✓, 2 ✓, 3 ✓ and 4 ✓ call the correct functions; choice 5 displays "Goodbye!" and breaks ✓ (7)

3.8 else ✓ displays "Invalid choice." ✓ (2)

3.9 Screenshot of the test run ✓ (1)

3.10 Program printed with line numbers ✓ (1)

```
# YourName - Question 3
scores = {"Alpha": [78, 85], "Bravo": [90], "Charlie": [65, 70, 80]}

def add_team():
    team = input("Team name: ")
    if team in scores:
        print(team, "is already registered.")
    else:
        scores[team] = []
        print(team, "added.")

def add_score():
    team = input("Team: ")
    if team not in scores:
        print(team, "is not registered.")
        return
    while True:
        try:
            score = int(input("Score (0-100): "))
            if 0 <= score <= 100:
                break
            print("Score must be from 0 to 100.")
        except ValueError:
            print("Please enter a whole number.")
    scores[team].append(score)
    print("Score saved.")

def leaderboard():
    ranked = sorted(scores, key=lambda t: sum(scores[t]), reverse=True)
    for place, team in enumerate(ranked, start=1):
        print(f"{place}. {team} - {sum(scores[team])} points ({len(scores[team])} runs)")
```

```
def team_average():
    team = input("Team: ")
    if team not in scores:
        print(team, "is not registered.")
    elif len(scores[team]) == 0:
        print(team, "has no scores yet.")
    else:
        print(team, "average:", round(sum(scores[team]) / len(scores[team]), 1))

while True:
    print("1 Add team  2 Add score  3 Leaderboard  4 Team average  5 Quit")
    choice = input("Choice: ")
    if choice == "1":
        add_team()
    elif choice == "2":
        add_score()
    elif choice == "3":
        leaderboard()
    elif choice == "4":
        team_average()
    elif choice == "5":
        print("Goodbye!")
        break
    else:
        print("Invalid choice.")
```

[40]

TOTAL: 100

END OF MARKING GUIDELINE